Far Western University

Central Campus School of Engineering

AI CHATBOT FOR NEPALI LAW AND GOVERNANCE – NITI-SATHI

A MINOR PROJECT REPORT

Submitted to

Department of Civil Engineering

In partial fulfillment of the requirement for the Degree of Bachelor's in Computer Engineering

Submitted by

Adarsh Joshi
Aryan Bhatt
Gajendra Pd. Badu
Mukesh Pant

July 27, 2025

Under the Supervision of

Er. Rohit Kumar Bisht

`

Far Western University

Central Campus School of Engineering

SUPERVISOR'S RECOMMENDATION

I hereby recommend that this project prepared under my supervision by **Rohit Kumar Bisht** entitled "**AI CHATBOT FOR NEPALI LAW AND GOVERNANCE – NITI-SATHI"** in partial fulfillment of the requirements for the degree of Bachelor of Computer Engineering is recommended for the final evaluation.

_____

Er. Rohit Kumar Bisht

SUPERVISOR

Assistant Professor

SchoolofEngineering

Mahendranagar, Kanchanpur

ABSTRACT

The project entitled “Niti-Sathi – An AI-Powered Legal Chatbot” is an innovative web-based platform designed to make Nepalese legal knowledge more accessible and understandable to the general public. Developed using a modern and lightweight tech stack including Streamlit (frontend), FastAPI (backend), LangChain, ChromaDB, and GPT-4.0 mini, the system leverages Retrieval-Augmented Generation (RAG) to respond to user queries with references to actual Nepali legal documents. The development approach followed the Rapid Application Development (RAD) model to allow quick prototyping and iterative updates based on testing and evolving project goals.

Niti-Sathi addresses the real-world problem of legal complexity, where understanding or finding the right legal information often requires expert consultation. The chatbot eliminates this barrier by providing a user-friendly interface for users to ask questions in English, and receive reliable, citation-based answers sourced from the Nepali constitution and other legal texts. The knowledge base is continuously updatable, and the system is designed to be scalable, cost-efficient, and informative.

Niti-Sathi achieves its objective of making legal governance more inclusive, bridging the gap between the people and the law through the effective use of artificial intelligence. By combining open-source technologies and AI capabilities, this project sets a foundation for future enhancements such as multilingual support, voice-based interaction, and integration with official legal portals.

Keywords: Streamlit, FastAPI, LangChain, ChromaDB, GPT-4.0 mini, RAG, Legal Chatbot, Nepal Constitution, AI for Governance.

ACKNOWLEDGEMENT

We are truly grateful to everyone who helped and supported us during the development of our project “Niti-Sathi: An AI-based Legal Chatbot”. Their valuable suggestions, encouragement, and guidance played a key role in helping us bring this project to life.

First and foremost, we would like to express our sincere gratitude to our supervisor, Er. Roshit Kumar Bisht, for his constant support, guidance, and motivation throughout the project. His insights and advice helped us stay focused and overcome the technical challenges we faced.

We also extend our heartfelt thanks to our friends from the law background, who shared their knowledge of the Nepali legal system and helped us better understand how laws work. Their help was crucial in shaping the content and knowledge base of our chatbot.

We are thankful to all our classmates and batchmates, who supported us with helpful feedback and motivation whenever we needed it.

Lastly, we deeply appreciate the overall support and cooperation we received from everyone around us. This project has been a great learning experience, and we thank all those who contributed to making “Niti-Sathi” a meaningful and impactful initiative.

TABLE OF CONTENTS

SUPERVISOR'S RECOMMENDATION ii

ABSTRACT iii

ACKNOWLEDGEMENT iv

LIST OF FIGURES vii

LIST OF TABLES viii

LIST OF LISTINGS ix

CHAPTER 1 INTRODUCTION 1

1.1 Introduction. 1

1.2 Problem statement 1

1.3 Objectives 2

1.4 Scope and Limitation 2

1.5 Report organization 3

CHAPTER 2 BACKGROUND STUDY AND LITERATURE REVIEW 4

2.1 Background study 4

2.2 Literature Review 5

CHAPTER 3 SYSTEM ANALYSIS AND DESIGN 7

3.1 System Analysis 7

3.1.1 Methodology 7

3.1.2 Requirement Analysis 8

3.1.3 Feasibility Analysis 10

3.1.4 System Flow Diagram 12

3.2 System Design 13

3.2.1 Architectural Design 13

3.2.2 Database Schema Design..... 14

3.2.3 Interface Design (UI Interface / Interface Structure Diagrams) 15

3.2.4 Algorithm and Mathematical Model.....16

CHAPTER 4 IMPLEMENTATION AND TESTING..... 18

4.1 Implementation 18

4.1.1 Tools Used..... 18

4.1.2 Implementation Details of Modules..... 20

4.2 Testing 23

4.2.1 Test Cases for Unit Testing..... 23

4.2.2 Test Cases for System Testing 25

CHAPTER 5 CONCLUSION AND FUTURE RECOMMENDATIONS.....26

5.1 Lesson learnt 26

5.2 Conclusion 27

5.3 Future Recommendations. 28

REFERENCES 29

LIST OF FIGURES

Figure 3.1: RAD Model 8

Figure 3.2: Use Case Diagram 9

Figure 3.3: Gantt Chart 11

Figure 3.4 Flow Chart of Chat Bot 12

Figure 3.5: Architectural Design 13

Figure 3.6: Database Schema Design 14

Figure 3.7: Interface Design of Front End 15

Figure 3.8: Simplified RAG Diagram 16

LIST OF TABLES

Table 1: Token Capacity 6

Table 2: Test Result of Document Upload.....23

Table 3: Test Result of Document Chunking and Embedding 24

Table 4: Test Result of Query Validation and Classifications 24

Table 5: Test Result of Legal Query Handling 25

LIST OF LISTINGS

Listing 4.1: Code Snippet to Create a Chat endpoint 20

Listing 4.2: Code Snippet to Create a File Upload endpoint 20

Listing 4.3: Code Snippet to Create a File Delete endpoint 21

Listing 4.4: Code Snippet to Create a File List endpoint 21

Listing 4.5: Code Snippet to List Document from Database 21

Listing 4.6: Code Snippet for describing RAG Chain in LLM 22

Listing 4.7: Code Snippet for setting retrieval tasks 22

Listing 4.8: Code Snippet to Create Table in Database 22

Listing 4.9: Code Snippet for Fetching Chat History 23

CHAPTER 1
INTRODUCTION

1.1 Introduction

Access to legal information in Nepal is often difficult due to complex legal language and unstructured digital resources. Most citizens lack awareness of their legal rights and struggle to find reliable and affordable legal assistance. Current legal resources are often limited to government websites that are hard to navigate.

To address this issue, we propose an AI chatbot that answers questions related to Nepali law and governance. This chatbot will be trained using legal texts from the Nepali Constitution and other governmental sources, making legal information accessible and user-friendly.

1.2 Problem Statement

Existing legal information systems are either difficult to navigate or lacks an AI-powered interaction. People often struggle to get precise legal answers without consulting expensive lawyers.

Challenges in the Current System:

1. Lack of easy-to-understand legal resources.
2. No AI-powered interactive solution for Nepali law.
3. Difficulty in searching for relevant laws quickly.

Our proposed AI chatbot will provide accurate legal information based on queries, helping citizens access Nepali law instantly in both Nepali and English.

1.3 Objectives

The main objectives of this project are:

1. To develop an AI-powered chatbot that answers legal questions based on Nepali law.
2. To Implement Retrieval-Augmented Generation for accurate legal responses.
3. To make Nepali legal documents searchable and user-friendly.
4. To support both Nepali and English for broader accessibility.

1.4 Scope and Limitation

The scope of Niti-Sathi, an AI-powered legal chatbot, is to provide a reliable and accessible platform for users to gain fundamental knowledge about Nepalese laws and governance. It is designed to simplify complex legal language and eliminate the need for extensive document searching or expensive legal consultations. Through a conversational interface, users can ask questions in English and receive responses backed by authentic legal references, using Retrieval-Augmented Generation (RAG) integrated with GPT-4.0 mini. The system supports real-time responses and aims to promote legal awareness among citizens by democratizing access to legal information.

The limitations of the current system include:

1. No multilingual support beyond Nepali and English, limiting accessibility for users speaking other regional languages.
2. Limited legal scope, currently focused only on pre-loaded documents such as the Constitution and selected acts.
3. Responses may lack depth, due to constraints of small open-source models and free-tier APIs.
4. No user account or history feature, which restricts personalization and follow-up interaction.
5. Hardware and performance limitations, as the project relies on limited CPU and memory resources and avoids large models like LLaMA due to deployment constraints.

1.5 Report Organization

The first part of this report, Chapter One, gives a clear introduction to the project. It includes the background, scope, limitations, objectives, and the real-life problem we are trying to solve making legal information more accessible to everyone using AI.

Chapter Two focuses on background research and literature review. It covers important concepts like language models, AI in legal systems, and Retrieval-Augmented Generation (RAG). It also reviews similar international projects such as DoNotPay and JusticeBot, which helped us shape the idea of "Niti-Sathi" for the Nepali context.

Chapter Three explains the system analysis and design. It describes what the system is supposed to do through functional and non-functional requirements. It also includes the use case and system flow diagrams that show how different users interact with the system. The overall design how the frontend, backend, AI model, and database work together is also explained here.

Chapter Four covers how the system was actually built and tested. The chatbot was developed using Streamlit for the frontend and FastAPI for the backend. ChromaDB is used to store and retrieve legal documents, and LangChain is used to connect the backend with GPT-4o-mini for accurate AI responses. This chapter also includes how the different parts were tested to make sure everything works smoothly.

Chapter Five concludes the report. It shares the final outcomes, what we learned during the development, and future possibilities like adding voice commands, mobile support, or real-time updates from legal sources with multiple languages.

CHAPTER 2
BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

The background study of Niti-Sathi, an AI-based legal information chatbot, focuses on understanding the problems people face while trying to get legal help and how technology can solve those problems. In Nepal, many people find it difficult to understand the Constitution, laws, and their own rights. This is not because they are not smart, but because legal documents are often written in difficult language and are hard to find. Most citizens don't know where to start or how to search for the right law.

This problem led us to think of a better way to provide legal information. We found that AI (Artificial Intelligence) can help people talk to a computer system like they talk to a person and get clear, helpful legal answers. Our project uses AI in a smart way. Instead of giving random answers, the system finds the correct law or rule from a legal database and shows that to the user in simple words. This process is called Retrieval-Augmented Generation (RAG), and it helps make the answers more accurate and trustworthy.

We also studied different tools and technologies that can be used to build this chatbot. We looked for tools that are free to use, easy to work with, and can run even on simple computers. Tools like Streamlit (for the website), FastAPI (for backend connection), LangChain, ChromaDB (for storing law documents), and GPT-4.0 mini (for AI responses) were selected because they are reliable and don't need very expensive machines.

During the study, we also looked at how other countries and platforms are using AI to help people understand the law. Many governments are starting to use similar chatbots to help citizens get quick legal help online. That gave us more confidence that Niti-Sathi is the right step toward making legal help more available in Nepal.

In summary, this background study helped us understand the real-life problems in getting legal help, the tools that can solve them, and how we can build a helpful, easy-to-use system for Nepali people.

2.2 Literature Review

Several AI-powered legal chatbots exist globally, such as DoNotPay (USA), Justice Bot (Canada) and LexiBot (UK). However, Nepal lacks a similar AI system tailored to it's legal framework. Studies show that NLP (Natural Language Processing) models like GPT-3.5, Rasa, and LangChain can be trained on legal documents to provide reliable legal assistance.

The paper "LEGALBOT - AI Law Advisor Chatbot", introduces an AI-driven chatbot designed to enhance legal accessibility through advanced Natural Language Processing (NLP) techniques. It provides users with legal guidance, terminology explanations, and document drafting support while maintaining an intuitive interface. The system integrates a structured legal knowledge base, enabling dynamic responses to user queries and improving legal research efficiency. LEGALBOT also offers lawyer contact details, expanding its role beyond advisory services. The study highlights the chatbot's potential in bridging the legal knowledge gap but acknowledges its limitations in replacing professional legal counsel. In the context of our AI Chatbot for Nepali Law project, this work provides valuable insights into structured legal data integration, NLP-driven interaction, and the role of AI in democratizing legal assistance. [1]

The paper "Nyaya Vaarta – AI Powered Legal Chatbot" presents the development of a judicial assistance chatbot built using the Botpress platform . Key contributions include a modular architecture allowing scalability, a response-generation module ensuring contextual accuracy, and a user-friendly web interface modeled after the DOJ website. Testing results indicate high response accuracy (~90%) and positive user feedback. Future improvements include multilingual support and expanding legal datasets. [2]

The article "How to customize LLMs like ChatGPT with your own data and documents" by Ben Dickson, discusses methods to tailor large language models (LLMs) such as ChatGPT to specific organizational needs by incorporating custom data through document embeddings. [3]

The proposed framework involves the following steps:

1. Extracting Text from Documents
2. Creating Document Embeddings
3. Storing Embeddings in a Vector Database

4. Developing the LLM Application

Regarding the alternative approach of fine-tuning LLMs, the article notes that while fine-tuning can yield good results without the need for context embedding, thereby reducing token and inference costs, it can be costly and complex. Using context embeddings is presented as a more accessible and cost-effective option, with the potential for fine-tuning as a future enhancement if a robust data collection pipeline is established. [4]

Table 1: Token Capacity

LLM	Token Limit	Estimated Word Count
Gpt-3.5	4096	3072
Gpt-3.5-turbo-16k	16385	12289
Gpt-4-32k	32768	24576
Gpt-4-turbo	128000	96000
Gpt-4o	128000	96000

CHAPTER 3
SYSTEM ANALYSIS AND DESIGN

3.1 System Analysis

System analysis means looking closely at a system to make it better. It involves planning for a new system to replace or work alongside the current one. During this process, we gather information, figure out what's wrong, and suggest ways to improve the system. The main goals of system analysis are to find problems, suggest solutions, and plan how to make those changes happen. System analysis is conducted considering the following objectives:

1. Consider the system concept for practicality.
2. Conduct economic and technical analyses.
3. Assign functions for hardware, software, people, database, and other factors.
4. Determine cost and scheduling restrictions.

3.1.1 Methodology

NITI-SATHI chatbot will be developed using the Rapid Application Development (RAD) model. RAD is a sequential software development approach that focuses on rapid prototyping, iterative improvements, and continuous feedback to achieve quick and efficient system deployment.We are using the RAD model to develop our AI chatbot for Nepalese law and governance in the following way:

1. Initial Rapid Development:

We are quickly building a basic chatbot that can answer general queries using OpenAI's API. The chatbot will initially provide legal information retrieval without a structured knowledge base.

2. Iterative Improvement & Feature Expansion:

In the next phases, we will integrate structured legal documents into the chatbot. The system will be enhanced to retrieve law-specific responses instead of just relying on general AI-generated answers. Features like multi-language support, improved accuracy, and citation-based responses will be added in each iteration.

3. Final Refinements & Optimization:

Once the core functionalities are working well, we will optimize performance and improve the chatbot's ability to handle multiple queries efficiently. The legal knowledge base will be regularly updated by the admin to ensure accuracy.

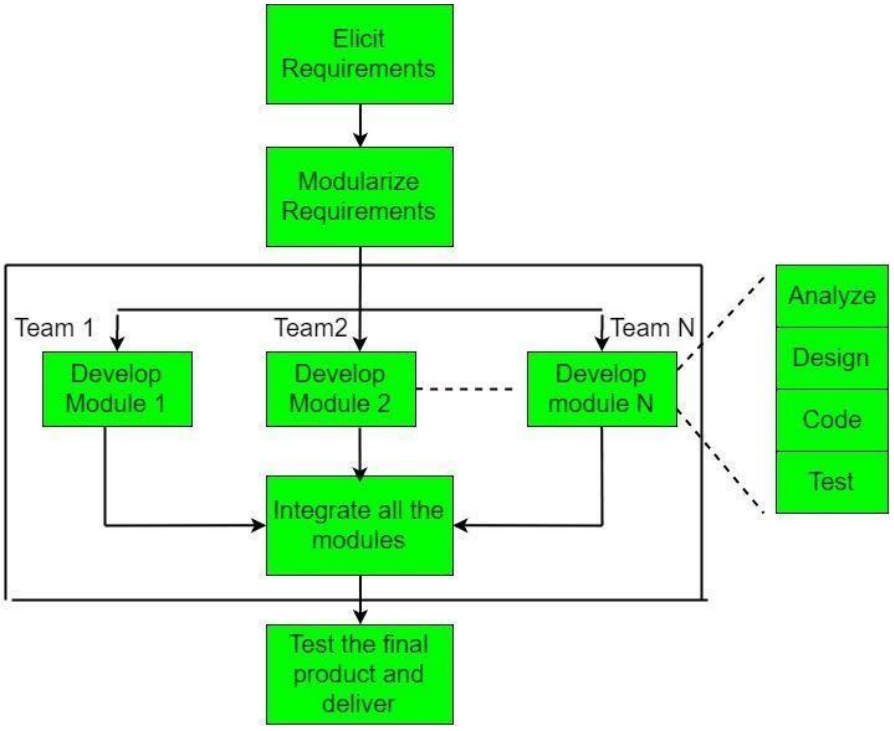

Figure 3.1: RAD Model

3.1.2 Requirement Analysis

i. Functional Requirements

Functional requirements specify what a system should do. They define specific functions, tasks, or actions that the system must be able to do in order to meet the needs of its users. These requirements focus on the system's behavior and capabilities, outlining how users will interact with it and what results they can expect. Functional requirements commonly involve user authentication, data input and processing, output generation, and system integration. By identifying these functional requirements, stakeholders can ensure that the system achieves the desired goals and offers the functionality required to meet user expectations.

The system must provide the following functionalities:

1. The chatbot should accept legal questions in Nepali and English.
2. It should retrieve relevant legal documents and generate responses.
3. Users should be able to interact via a web-based UI.
4. The chatbot should handle multiple users simultaneously.
5. It should provide references to legal documents in responses

Functional requirements can be expressed in the form of Use Cases since they demonstrate externally evident functional behavior.

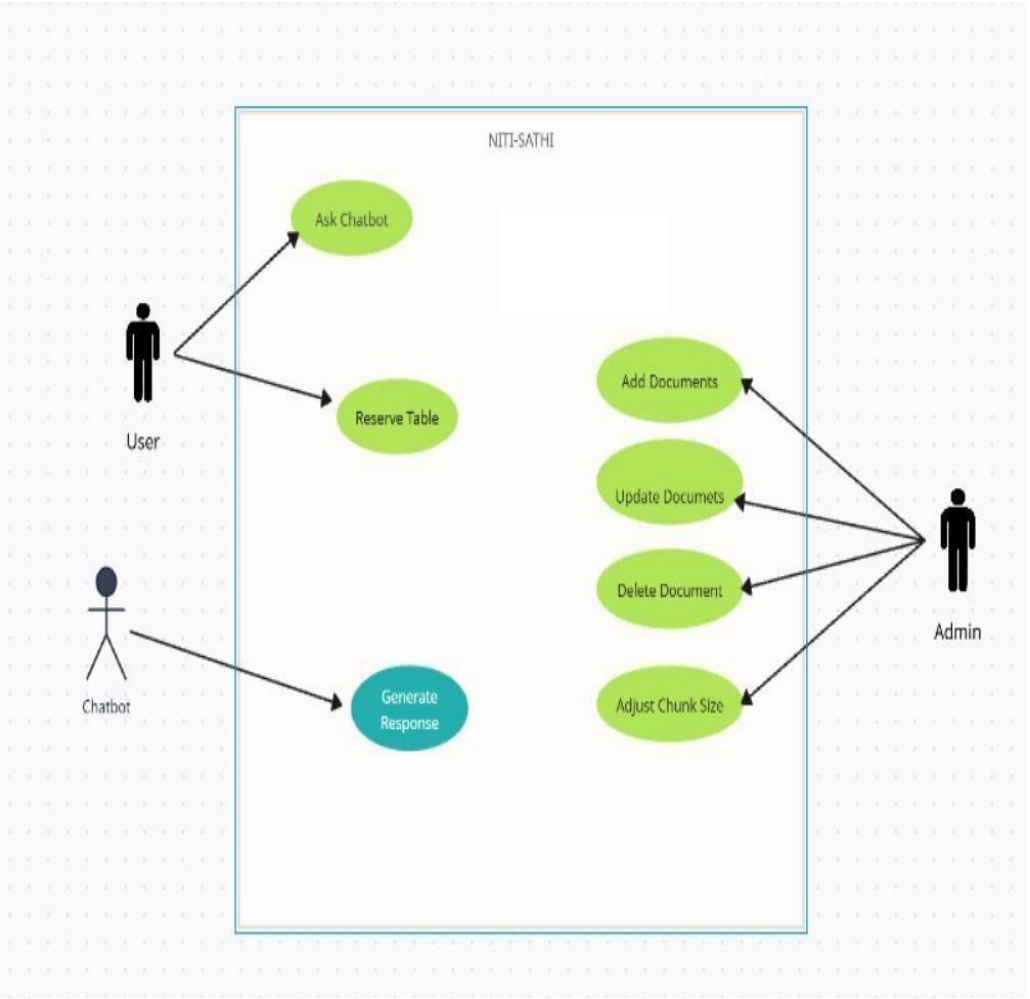

Figure 3.2: Use Case Diagram

ii. Non-Functional Requirements

Non-functional requirements define parameters for the system's performance, usability, dependability, and security. Unlike functional requirements, which describe what the system does, non-functional requirements specify how the system should act or operate. These criteria include reaction time, scalability, availability, accessibility, maintainability, and security features. Non-functional requirements are critical for ensuring that the system runs smoothly and fulfills the overall quality standards set by users and stakeholders. They outline how to evaluate the system's performance and ensure that it provides a satisfying user experience while adhering to regulatory and industry requirements.

This system provides the following nonfunctional requirements:

1. The chatbot must be highly available and scalable.
2. Responses should be generated within 3 seconds.
3. The system should be secure and prevent unauthorized access.
4. It should support future expansions, such as additional legal domains.
5. The web interface must be user-friendly and responsive.

3.1.3 Feasibility Analysis

i. Technical Feasibility

To ensure a smooth development process, the system will utilize stable and well-documented technologies. The chatbot will be built using Streamlit for the frontend, FastAPI (Python) for the backend, and SQLite as the database for secure and efficient data storage. The AI model will be based on OpenAI's API with Retrieval- Augmented Generation (RAG) for accurate legal responses. The project will minimize technical disruptions by leveraging existing frameworks, ensuring system stability and efficiency. Given the infrastructure, software, and technology that we used, the project is simply suitable.

To deploy the application, these minimum requirements need to be fulfilled.

Hardware Requirements

- Server: Minimum 8-core CPU, 8GB RAM (for hosting AI and database).
- Client Devices: Any device with a web browser and stable internet.

Software Requirements

- Frontend: Streanlit (Python)
- Backend: FastAPI (Python)
- Database: ChromaDB
- AI Model: OpenAI API with Retrieval-Augmented Generation (RAG)
- Hosting: Localhost

ii. Operational Feasibility

The chatbot is designed to be user-friendly and easy to adapt. It will feature a simple web interface that allows users to enter legal queries in Nepali and English without any technical knowledge. The system will be scalable, ensuring that it can handle multiple queries efficiently and be expanded in the future. Proper documentation and regular updates will ensure smooth operation and maintenance.

These key operational features:

- Simple and intuitive interface for ease of use
- Automatic updates for legal documents
- Scalable architecture for future enhancements
- Multi-user support to handle concurrent requests

iii. Economic Feasibility

This project is highly cost-effective, relying on open-source technologies and free-tier cloud services for deployment. The chatbot will be hosted using Vercel for the frontend and Render for the backend, eliminating the need for expensive infrastructure. The AI model will use a pay-as-you-go API, reducing upfront costs. Since no additional hardware is required, development and operational costs remain minimal.

Key Financial Considerations:

- Development Costs: Minimal due to open-source tools
- Hosting Costs: Free-tier hosting services for initial deployment
- Maintenance Costs: Low, with occasional updates to the legal database

iv. Schedule Feasibility

This includes the project schedule and all the time that is allocated for their completion.

The Gantt Chart of the entire project is shown below:

Figure 3.3: Gantt Chart

3.1.4 System Flow Diagram

The system flow describes how the chatbot processes user queries using AI and Retrieval- Augmented Generation (RAG) to provide accurate legal information.

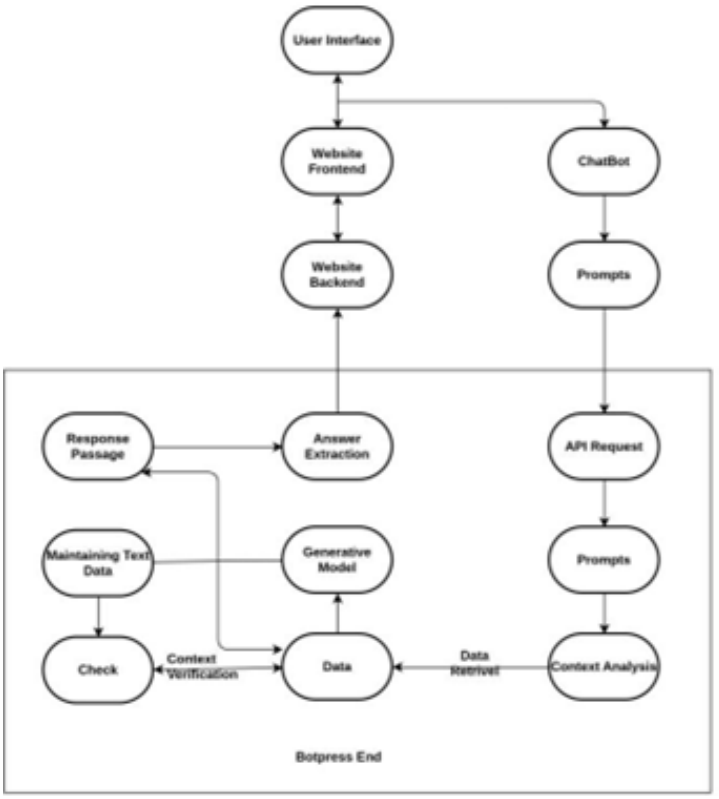

Figure 3.4: Flow Chart of Chat Bot

When a user submits a query via the frontend (web interface), it is sent to the backend (Flask API). The backend checks if legal references are needed and retrieves relevant documents from the knowledge base if required.

Using the RAG model, the retrieved legal text is combined with the query and processed by OpenAI's API. This ensures responses are based on actual legal sources rather than just AI-generated text. The response is refined, formatted, and includes proper citations for clarity and accuracy.

Finally, the backend sends the response to the frontend, where the user receives a structured and reliable legal answer, making legal information easily accessible and verifiable.

3.2 System Design

3.2.1 Architectural Design

Architectural Design is a process for identifying the sub-systems making up a system and the framework for sub-system control and communication.

Figure 3.5: Architectural Design

3.2.2 Database Schema Design

A database named rag_app is created and it contains two tables, each for storing document and application logs. document_store table contains various fields with id being its unique identifier. applicayion_log table also contain the id as primary key and session_id for identifying chat sessions.

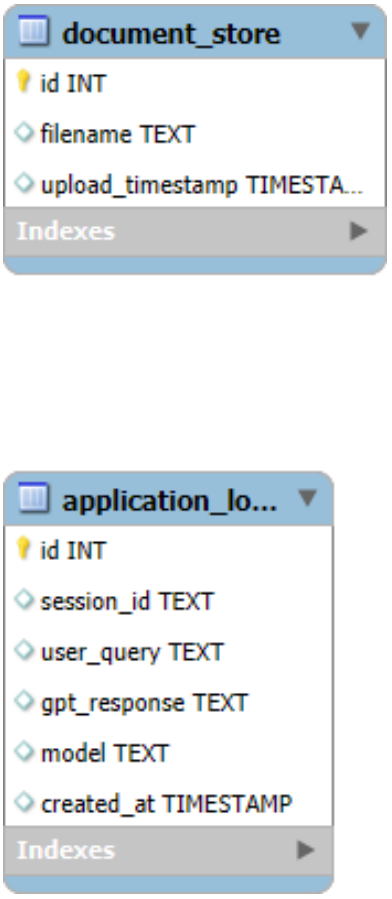

Figure 3.6: Database Schema Design

3.2.3 Interface Design (UI Interface / Interface Structure Diagrams)

The user interface of the application has been designed using Streamlit in python. Streamlit is an open-source Python library used to build simple and interactive web applications easily and quickly.

Front End Page:

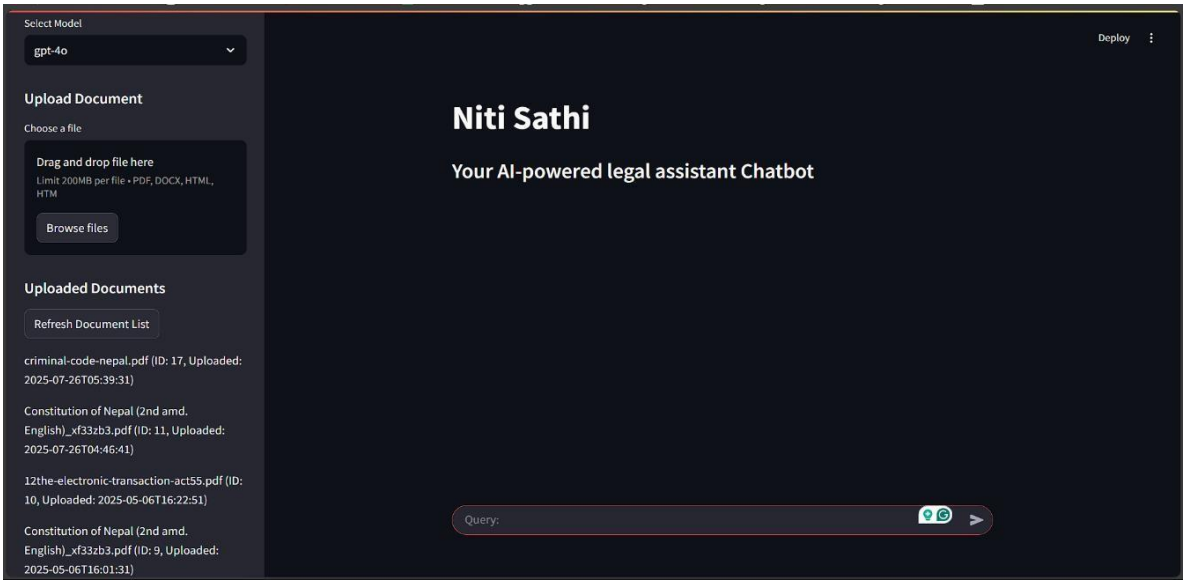

Figure 3.7: Interface Design of Front End

3.2.4 Algorithm & Mathematical Model

Algorithm Used in AI Chatbot:

RAG is an AI technique that combines information retrieval and text generation to provide accurate and context-aware responses. Instead of relying solely on pre-trained knowledge, the chatbot dynamically retrieves relevant legal documents from a structured database before generating responses.

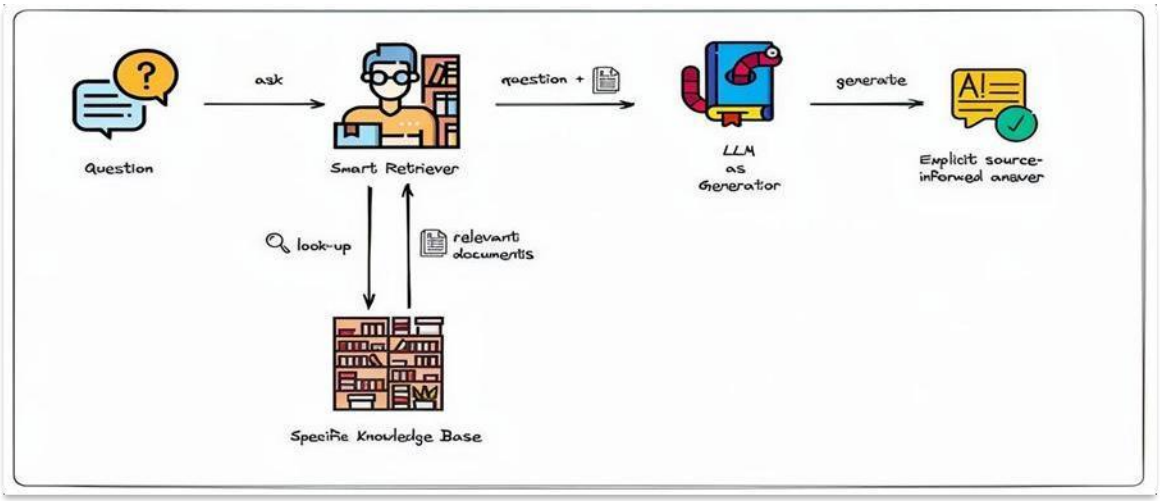

Figure 3.8: Simplified RAG Diagram

How RAG Works in This System:

1. User Query Processing:
   a) The user enters a legal question.
   b) The system converts the query into an embedding (numerical representation) for better search accuracy.
2. Document Retrieval:
   a) The AI searches through Nepalese legal texts stored in the database.
   b) It ranks the most relevant legal documents using similarity algorithms such as Cosine Similarity.
3. Response Generation:
   a) The chatbot uses the retrieved documents as a knowledge source.
   b) OpenAI's language model generates a response based on the extracted legal references.
4. Final Output:
   a) The chatbot presents a structured response to the user in a conversational

format.

b) It includes citations to official legal documents for user verification.

Mathematical Model:

Cosine Similarity is a fundamental technique used in **Retrieval-Augmented Generation (RAG)** to measure the relevance between a user's query and stored legal documents. It helps the AI system find the most relevant legal texts before generating a response.

Formula for Cosine Similarity:

$$Similarity\ (A, B) = \frac{A \cdot B}{|A| \times |B|} \qquad \ldots (i)$$

Were,

- A = Vector representation of the user query
- B = Vector representation of a legal document
- A · B = Dot product of the two vectors
- |A| and |B| = Magnitude (length) of the vectors

How Cosine Similarity Works in RAG:

1. Query Embedding: The user's question is converted into a vector representation using an AI embedding model.
2. Document Retrieval: The system compares the query vector with document vectors stored in the database and ranks them based on similarity.
3. Response Generation: The chatbot extracts relevant legal sections from the retrieved documents and generates an AI-powered response.

CHAPTER 4
IMPLEMENTATION AND TESTING

4.1 Implementation

4.1.1 Tools Used

Front-End Tools

1. Streamlit

Streamlit is an open-source Python library used to build simple and interactive web applications easily and quickly. In this project, it is used to design the frontend interface where users can enter their legal queries and view responses. It helps present the chatbot in a clean and user-friendly format with minimal coding.

Backend-Tools

1. FastAPI

FastAPI is a modern, high-performance Python web framework used to build backend APIs. In this system, it handles user queries from the frontend, communicates with the AI model, manages data retrieval from the database, and sends back accurate responses. It's fast, easy to use, and supports asynchronous operations.

2. LangChain

LangChain is a powerful framework for developing AI applications that use large language models (LLMs). In this project, LangChain helps connect the OpenAI LLM (GPT-4o mini) with our legal knowledge base. It handles the RAG (Retrieval-Augmented Generation) process—retrieving relevant documents and passing them to the LLM for accurate response generation.

3. Hugging Face Transformers (Embeddings)

We use Hugging Face's transformer models to convert legal texts and user queries into embeddings—numerical vector representations. These embeddings are essential for comparing and finding the most relevant legal documents. It enables the AI to understand the meaning of queries and match them with the right legal data.

Database

1. ChromaDB

ChromaDB is an open-source vector database designed for storing and retrieving embeddings efficiently. It stores the legal documents in vector form and helps retrieve the most relevant ones when a user submits a query. It plays a key role in the document retrieval step of the RAG process.

Documentation Tool

1. MS Word

MS Word is used solely for creating comprehensive and well-organized project documentation. MS Word is used throughout for writing and editing the documentation of this project.

Diagrammatic Tool

1. Draw.io

Draw.io is heavily utilized in this project to generate a variety of diagrams, such as data flow, entity-relationship, and use case diagrams. It is used to diagrammatically represent complex system requirements.

4.1.2 Implementation Details of Modules

The system's modules are shown below:

- **Posts Module**

```python
@app.post("/chat", response_model=QueryResponse)
def chat(query_input: QueryInput):
    session_id = query_input.session_id
    logging.info(f"Session ID: {session_id}, User Query: {query_input.question}, Model: {query_input.model.value}")
    if not session_id:
        session_id = str(uuid.uuid4())

    chat_history = get_chat_history(session_id)
    rag_chain = get_rag_chain(query_input.model.value)
    answer = rag_chain.invoke({
        "input": query_input.question,
        "chat_history": chat_history
    })['answer']

    insert_application_logs(session_id, query_input.question, answer, query_input.model.value)
    logging.info(f"Session ID: {session_id}, AI Response: {answer}")
    return QueryResponse(answer=answer, session_id=session_id, model=query_input.model)
```

Listing 4.1: Code Snippet to Create a Chat endpoint

This code snippet defines a chatbot API endpoint that handles user queries in an interactive session. It logs user input, retrieves previous chat history, and uses a selected AI model to generate a relevant response. If no session ID is provided, a new one is created to maintain conversation continuity. The response is then returned along with session details and logged for future reference.

```python
@app.post("/upload-doc")
def upload_and_index_document(file: UploadFile = File(...)):
    allowed_extensions = ['.pdf', '.docx', '.html']
    file_extension = os.path.splitext(file.filename)[1].lower()

    if file_extension not in allowed_extensions:
        raise HTTPException(status_code=400, detail=f"Unsupported file type. Allowed types are: {', '.join(allowed_extensions)}")
    temp_file_path = f"temp_{file.filename}"

    try:
        # Save the uploaded file to a temporary file
        with open(temp_file_path, "wb") as buffer:
            shutil.copyfileobj(file.file, buffer)

        file_id = insert_document_record(file.filename)
        success = index_document_to_chroma(temp_file_path, file_id)

        if success:
            return {"message": f"File {file.filename} has been successfully uploaded and indexed.", "file_id": file_id}
        else:
            delete_document_record(file_id)
            raise HTTPException(status_code=500, detail=f"Failed to index {file.filename}.")
    finally:
        if os.path.exists(temp_file_path):
            os.remove(temp_file_path)
```

Listing 4.2: Code Snippet to Create a File Upload endpoint

This code snippet is part of an API endpoint that handles file uploads. It checks if the uploaded file has an allowed extension (like PDF, DOCX, or HTML) and then

temporarily saves the file. The code attempts to store information about the file in a database and index it for further use. If the operation succeeds, it returns a success message with a file ID; otherwise, it cleans up by deleting the temporary file and returns an error message.

```
@app.post("/delete-doc")
def delete_document(request: DeleteFileRequest):
    # Delete from Chroma
    chroma_delete_success = delete_doc_from_chroma(request.file_id)

    if chroma_delete_success:
        # If successfully deleted from Chroma, delete from our database
        db_delete_success = delete_document_record(request.file_id)
        if db_delete_success:
            return {"message": f"Successfully deleted document with file_id {request.file_id} from the system."}
        else:
            return {"error": f"Deleted from Chroma but failed to delete document with file_id {request.file_id} from the database."}
    else:
        return {"error": f"Failed to delete document with file_id {request.file_id} from Chroma."}
```

Listing 4.3: Code Snippet to Create a File Delete endpoint

This code snippet defines an API endpoint for deleting documents. It first attempts to remove a document from a storage system called Chroma and then deletes its record from a database. If both operations succeed, it returns a success message with the file ID. If either step fails, it responds with an error message indicating where the deletion process failed.

```
@app.get("/list-docs", response_model=list[DocumentInfo])
def list_documents():
    return get_all_documents()
```

Listing 4.4: Code Snippet to Create a File List endpoint

This code lists all the existing documents present.

```
def get_all_documents():
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('SELECT id, filename, upload_timestamp FROM document_store ORDER BY upload_timestamp DESC')
    documents = cursor.fetchall()
    conn.close()
    return [dict(doc) for doc in documents]
```

Listing 4.5: Code Snippet to List Document from Database

This code snippet defines a function that retrieves all stored documents from a database. It fetches details like the document ID, filename, and upload time, arranging them in descending order of upload.

```python
def get_rag_chain(model="gpt-4o-mini"):
    llm = ChatOpenAI(model=model, api_key=OPENAI_API_KEY)
    history_aware_retriever = create_history_aware_retriever(llm, retriever, contextualize_q_prompt)
    question_answer_chain = create_stuff_documents_chain(llm, qa_prompt)
    rag_chain = create_retrieval_chain(history_aware_retriever, question_answer_chain)
    return rag_chain
```

Listing 4.6: Code Snippet for describing RAG Chain in LLM

This code snippet defines a function that sets up a retrieval-augmented generation (RAG) chain using a language model. It combines a retriever that understands conversation history with a question-answering chain to handle user queries effectively. The function then returns the complete RAG chain for use in generating responses.

```python
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200, length_function=len)

# Initialize HuggingFace embeddings without API token
embedding_function = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2",
    model_kwargs={"device": "cpu"}  # Use CPU; change to "cuda" if GPU is available
)

vectorstore = Chroma(persist_directory="./chroma_db", embedding_function=embedding_function)
```

Listing 4.7: Code Snippet for setting retrieval tasks

This code snippet sets up a system for processing and storing text data for retrieval tasks. It splits large text into smaller overlapping chunks, creates embeddings using a pre-trained HuggingFace model, and stores these embeddings in a local Chroma database. This setup enables efficient text search and retrieval.

```python
def create_application_logs():
    conn = get_db_connection()
    conn.execute('''CREATE TABLE IF NOT EXISTS application_logs
                    (id INTEGER PRIMARY KEY AUTOINCREMENT,
                     session_id TEXT,
                     user_query TEXT,
                     gpt_response TEXT,
                     model TEXT,
                     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP)''')
    conn.close()
```

Listing 4.8: Code Snippet to Create Table in Database

This code snippet defines a function to create a database table for storing application logs if it doesn't already exist. The table is designed to keep records of user queries, AI responses, the model used, and timestamps for each entry.

```python
def get_chat_history(session_id):
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute('SELECT user_query, gpt_response FROM application_logs WHERE session_id = ? ORDER BY created_at', (session_id,))
    messages = []
    for row in cursor.fetchall():
        messages.extend([
            {"role": "human", "content": row['user_query']},
            {"role": "ai", "content": row['gpt_response']}
        ])
    conn.close()
    return messages
```

Listing 4.9: Code Snippet for Fetching Chat History

This code snippet defines a function that retrieves the chat history for a given session from the database. It collects all user queries and AI responses in chronological order and formats them as a list of messages with roles like "human" and "ai." Finally, it returns the complete conversation history.

4.2 Testing

Various functions and components were tested.

4.2.1 Test Cases for Unit Testing

Document Upload

Table 1: Test Result of Document Upload

Test Case Id	Test Scenario	Test Steps	Test Data	Expected Result	Pass / Fail
UT01	Uploa d a valid PDF	Upload a `.pdf` file via admin dashboard	`law1.pdf`	System confirms upload and stores document	Passed
UT02	Uploa d invalid file format	Upload `.jpg` via upload form	`image.jpg`	Error message: “Not supported format.”	Passed

Document Chunking & Embedding

Table 2: Test Result of Document Chunking and Embedding

Test Case Id	Test Scenario	Test Steps	Test Data	Expected Result	Pass / Fail
UT03	Docu ment chunking	Trigger document splitting	Uploaded PDF	Document is chunked into clean segments	Passed
UT04	Generat e embeddi ng	Apply embedding model on a chunk	Chunk text	Embedding vector is returned successfully	Passed
UT05	Save to Vector DB	Store vector in ChromaDB	Vector from chunk	Vector is saved and indexed correctly	Passed

Query Processing

Table 3: Test Result of Query Validation and Classification

Test Case Id	Test Scenario	Test Steps	Test Data	Expected Result	Pass / Fail
UT06	Empty user query	Submit blank message	(blank input)	Message: "Please enter your query."	Passed
UT07	Irrelev ant query	Ask unrelated question	Tell me a joke	Response: "No legal reference found for this query."	Passed
UT08	API failure handling	Disconnect backend/API	(simulated failure)	Message: "System temporarily unavailable. Please try again later."	Passed

4.2.2 Test Cases for System Testing

Table 4: Test Result of Legal Query Handling

Test Case Id	Test Scenario	Test Steps	Expected Result	Pass / Fail
ST01	Full chatbot query flow	User enters legal query in English/Nepali	Structured and relevant legal response	Passed
ST02	Doc upload → Ask → Respond	Upload law → Ask related query	Chatbot responds using newly uploaded doc	Passed
ST03	Unrelated question handling	Ask off-topic query	No legal reference found.	Passed
ST04	Re-query deleted document	Delete doc → Ask from it again	System says no matching document found	Passed
ST05	Response from updated document	Replace document → Ask updated content	Chatbot reflects new content	Passed
ST06	Backend/API failure fallback	Simulate ChromaDB or OpenAI API outage	Error: "Sorry, something went wrong. Try again later."	Passed

CHAPTER 5
CONCLUSION AND FUTURE RECOMMENDATIONS

5.1 Lesson Learnt

Throughout the development of Niti-Sathi, we faced a range of challenges that helped us understand the practical limitations of building an AI chatbot using LLMs and RAG. Some key lessons include:

1. Implementing LLMs with RAG is Complex: As beginners, integrating Retrieval-Augmented Generation (RAG) with a language model required a deep understanding of prompt engineering, document chunking, embeddings, and vector search. It was more complex than expected and demanded careful planning.
2. Limited Computational Resources: We had no access to high-end CPUs or GPUs, which restricted us from using larger and more powerful local models like Ollama, Mistral, or Llama2. This forced us to rely on cloud-based services or lightweight alternatives.
3. Paid APIs and Models Are Often Required: Many powerful models and accurate embedding generators (like OpenAI's Ada or Cohere embeddings) are behind paywalls. Since we operated on a limited budget, we used open-source or free-tier models, which limited the chatbot's response depth and contextual understanding.
4. Token and Context Limitations: Free-tier APIs like OpenAI GPT or Hugging Face often restrict the number of tokens allowed in a single query or response. This made it difficult to pass both long legal documents and user queries together, impacting response completeness.
5. Lack of Multilingual Support in Free Models: Although we intended to support both Nepali and English, most freely available models did not understand or generate Nepali text well. Without paid or multilingual LLMs like GPT-4 full version or Gemini Pro, providing accurate bilingual output remained a major challenge.
6. Performance vs. Capability Trade-offs: Due to constraints, we had to compromise on some advanced features such as real-time updates, scalable databases, or interactive UI, to maintain a fast, stable system with basic functionality.
7. Value of Iterative Testing: Through continuous testing, we realized how changing chunk sizes, embedding strategies, and prompt formatting could significantly

impact the relevance and correctness of chatbot responses.

5.2 Conclusion

In conclusion, developing Niti-Sathi, an AI-powered legal chatbot built using Streamlit (frontend), FastAPI (backend), LangChain, ChromaDB, and GPT-4.0 mini, has been an insightful and rewarding journey. The goal was to create a simple, accessible platform that could provide legal information to Nepalese users without needing deep technical knowledge.

While the project faced technical and financial challenges, we successfully implemented a working prototype that uses Retrieval-Augmented Generation to answer legal queries with reliable sources. It bridges the gap between ordinary people and complex legal language by making information more interactive and easier to access.

This project not only fulfilled our academic goals but also opened doors to how AI can make governance more transparent and inclusive.

5.3 Future Recommendations

Though the current system works well, there are several areas for future improvements:

1. Multilingual Support: Add support for multiple languages and dialects to make the chatbot usable by a wider population in Nepal.
2. Mobile Version: Develop a lightweight mobile-friendly version so users in rural areas or with low-end devices can also benefit.
3. User Personalization: Introduce login systems where users can save their queries, preferences, and get tailored legal help over time.
4. Live Legal Updates: Sync with government portals so that the legal database remains up-to-date with recent laws and amendments.
5. Audio and Accessibility Features: Allow voice-based input and response for visually impaired or low-literacy users.
6. Upgrade to Better Models: Once resources allow, migrate to more powerful LLMs and better embedding models for more thoughtful and human-like responses.

REFERENCES

[1] B. Pardhi, S. Koli, V. Khanzode, and A. S. Raut, "LEGALBOT - AI law advisor chatbot," *Int. J. Novel Res. Develop.*, vol. 9, no. 4, pp. 1–10, Apr. 2024. Available: https://www.ijnrd.org/papers/IJNRD2404730.pdf.

[2] M. Singhal, U. Mathur, U. Agarwal, Y. Shirmal, and C. Upadhyay, "Nyaya Vaarta – AI powered legal chatbot," *Int. J. Comput. Technol.*, vol. 11, no. 6, pp. 105–110, Dec. 2024. Available: https://ijctjournal.org/wp-content/uploads/2024/12/ijct-v11i6p14.pdf.

[3] B. Dickson. (2023, May 1). "How to customize LLMs like ChatGPT with your own data and documents" [Online]. Available: https://bdtechtalks.com/2023/05/01/customize- chatgpt-llm-embeddings/.

[4] M. Rozière, B. Hunyh, A. Lachaux, G. Lample, M. Kim, L. Deleu, L. Zaidi, T. Lavril, and A. Synnaeve, "Code Llama: Open foundation models for code," *arXiv preprint arXiv:2303.13217*, Aug. 2023. Available: https://arxiv.org/pdf/2303.13217.